

## Sección 1: Bases de Datos NoSQL y Relacionales

1) Base de datos: Si. Agrupa a las colecciones

Tablas/Relaciones: No. Existen las colecciones, que son un conjunto de documentos. A las bases de datos NoSQL no se les define un esquema

Fila/Tupla: No. Hay documentos, que pueden tener datos anidados complejos (incluso otros documentos).

Columna: No. Hay campos de un documento

2) No existen las Claves Foráneas en MongoDB, sino que las relaciones se establecen a partir de anidar documentos o de referenciarlos. Si el array de referencias que se genera es muy grande, entonces se puede invertir la referencia

3) Se crean índices para hacer más eficientes las queries. Sin indicios, se deberían recorrer todos los documentos en una colección para retornar un resultado. Si existe un índice, MongoDB lo utiliza para limitar la cantidad de documentos que debe escanear.

Añadir índices tiene un impacto negativo en las operaciones de **escritura**, principalmente en colecciones con un ratio alto de escrituras-lecturas.

Tipos de índices en MongoDB:

- Single Field: agrupan y ordenan información en base a un solo campo de cada documento de la colección

- Compound: agrupan y ordenan información por dos o más campos de cada documento en una colección. Primero se agrupa por el primer campo y luego por cada uno del resto.

- Multikey: agrupan y ordenan información almacenada en arreglos. No hay que especificar explícitamente que el índice es de este tipo; MongoDB detecta que el índice se creó sobre un arreglo

- Geoespaciales: mejoran performance en queries realizadas sobre información con coordenadas geoespaciales.

- Texto: mejora performance en queries de búsqueda de texto sobre campos string

- Hashed: indexan el hash del valor de un campo

4) En MongoDB una vista es un objeto de sólo lectura al que se le pueden realizar queries. No se persisten en disco, sino que su contenido se computa **a demanda** cuando el cliente consulta la vista.

2 tipos de vista:

- Estándar: se computan al leerlas y no se guardan en disco.

- Vista materializada: son escritas y leídas en el disco.

Índices:

- Estándar: usan los de la colección subyacente. No pueden crearse índices nuevos

- Materializada: se pueden crear directamente sobre las vistas dado que se guardan en disco

Performance: las vistas materializadas tienen un mejor rendimiento en la lectura dado que se leen del disco en lugar de computarse como parte de la query

Casos de uso:

- Estándar: ocultar campos sensibles
- Materializadas: acelerar consultas costosas que se realizan frecuentemente

5) Si, MongoDB permite realizar validación de esquema especificando reglas que deben cumplir los campos de los documentos de una colección.

Por defecto es flexible. Documentos pueden tener distintos campos de distinto tipo ->

**\*\*Flexible\*\***

Recomendado en aplicaciones en etapas avanzadas, cuando ya se conocen restricciones en los campos.

6) Las operaciones a nivel de un documento son atómicas, incluso si los documentos están anidados. Se pueden usar transacciones para realizar de manera unitaria operaciones sobre múltiples documentos o colecciones.

En la mayoría de los casos, poseer el modelo de datos desnormalizado (anidamiento de documentos y arreglos) es más eficiente que usar transacciones distribuidas

8)

Route -> Stops: Embebido

Purchase -> Review: Embebido

Tourguide/DriverUser -> Route: Referencia.

## Sección 2: Operaciones CRUD básicas

```
test> use tours
switched to db tours
```

9)i)

```
tours> db.recorridos.insertOne({
...   nombre: "City Tour",
...   precio: 200,
...   stops: [
...     "Diagonal Norte",
...     "Avenida de Mayo",
...     "Plaza del Congreso"
...   ],
...   totalKm: 5
... })
{
  acknowledged: true,
  insertedId: ObjectId('6836263b10d3b75e38c59f35')
}
```

ii)

```
tours> db.recorridos.find()
[
  {
    _id: ObjectId('6836263b10d3b75e38c59f35'),
    nombre: 'City Tour',
    precio: 200,
    stops: [ 'Diagonal Norte', 'Avenida de Mayo', 'Plaza del Congreso' ],
    totalKm: 5
  }
]
```

10)

```
mongoimport --db=tours --collection=recorridos --file=Descargas/material_adicional_1.json --jsonArray
2025-05-27T18:16:16.609-0300 connected to: mongodb://localhost/
2025-05-27T18:16:16.610-0300 14 document(s) imported successfully. 0 document(s) failed to import.
```

i)

```
tours> db.recorridos.updateOne({nombre:"Cultural Odyssey"},
... {$set: {totalKm: 12}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

ii)

```
tours> db.recorridos.updateOne(
... {nombre: "Delta Tour"},
... {$push: {stops: "Tigre"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

iii)

```
tours> db.recorridos.updateMany(
... {},
... {$mul: {precio: 1.1} }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 15,
  modifiedCount: 15,
  upsertedCount: 0
}
```

iv)

```
tours> db.recorridos.deleteMany( {nombre: "Temporal Route"} )
{ acknowledged: true, deletedCount: 1 }
```

v)

```
tours> db.recorridos.updateOne( { nombre: "Urban Exploration"},
... {$push: {tags: "Gastronomia"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

11)

- i) db.recorridos.find({ nombre: "Museum Tour" })
- ii) db.recorridos.find({ precio: { \$gt: 600 } })
- iii) db.recorridos.find({ precio: { \$gt: 500 }, totalKm: { \$gt: 10 } })
- iv) db.recorridos.find({ stops: "San Telmo" })
- v) db.recorridos.find({ \$and: [ { stops: "Recoleta" }, { stops: { \$nin: ["Plaza Italia"] } } ] })

vi) db.recorridos.find({ stops: "Delta", precio: { \$lt: 500 } }, { nombre: 1, totalKm: 1, \_id: 0 })  
vii) db.recorridos.find({ stops: { \$all: ["San Telmo", "Recoleta", "Avenida de Mayo"] } })  
viii) db.recorridos.find({ "stops.5": { \$exists: true } }, { nombre: 1, \_id: 0 })  
ix) db.recorridos.find({ totalKm: { \$exists: false } })  
x) db.recorridos.find({ stops: { \$regex: /museo/i } }, { nombre: 1, stops: 1, \_id: 0 })  
xi) db.recorridos.countDocuments()

## Sección 3: Aggregation Framework

12)

```
tours2> load("Descargas/generator1.js")
DeprecationWarning: Collection.insert() is deprecated. Use insertOne, insertMany, or bulkWrite.
true
tours2> show collections
route
stop
```

13)

i)

```
tours2> db.route.aggregate([
... {
...   $sample: { size: 5 }
... }
... ])
```

ii)

```
db.route.aggregate([
  { $sample: { size: 5 } },
  { $lookup: {
    from: "stop",
    localField: "stops",
    foreignField: "code",
    as: "stops_info"
  } }
])
```

iii)

```
db.route.aggregate([
  { $match: { price: { $gte: 900 } } },
  { $lookup: {
```

```

        from: "stop",
        localField: "stops",
        foreignField: "code",
        as: "stops_info"
    } }
])

```

```

iv)
db.route.aggregate([
  { $match: { "stops.4": { $exists: true } } },
  { $lookup: {
    from: "stop",
    localField: "stops",
    foreignField: "code",
    as: "stops_info"
  } }
])

```

```

v)
db.route.aggregate([
  { $match: { name: { $regex: "111" } } }
])

```

```

vi)
db.route.aggregate([
  { $match: { name: "route100" } },
  { $lookup: {
    from: "stop",
    localField: "stops",
    foreignField: "code",
    as: "stops_info"
  } },
  { $project: { stops_info: 1, _id: 0 } }
])

```

```

vii)
db.route.aggregate([
  { $unwind: "$stops" },
  { $group: { _id: "$stops", apariciones: { $sum: 1 } } },
  { $sort: { apariciones: -1 } },
  { $limit: 1 },
  { $lookup: {
    from: "stop",
    localField: "_id",
    foreignField: "code",
    as: "stop_info"
  } },
  { $unwind: "$stop_info" },

```

```
    { $project: { _id: 0, apariciones: 1, stop_info: 1 } }  
  ])
```

viii)

```
db.route.aggregate([  
  { $match: { price: { $lt: 150 } } },  
  { $addFields: { cantidadStops: { $size: "$stops" } } },  
  { $out: "rutas_economicas" }  
])
```

ix)

```
db.stop.aggregate([  
  { $lookup: {  
    from: "route",  
    localField: "code",  
    foreignField: "stops",  
    as: "rutas_que_lo_incluyen"  
  } },  
  { $unwind: "$rutas_que_lo_incluyen" },  
  { $group: {  
    _id: "$code",  
    nombreStop: { $first: "$name" },  
    precioPromedio: { $avg: "$rutas_que_lo_incluyen.price" }  
  } },  
  { $project: {  
    _id: 0,  
    codigoStop: "$_id",  
    nombreStop: 1,  
    precioPromedio: { $round: ["$precioPromedio", 2] }  
  } }  
])
```